

Code Style Specification

OpenBSD KNF layout with camelCase and Hungarian notation

1 Scope and precedence

This document defines a C coding style that keeps the *layout* rules of OpenBSD's Kernel Normal Form (**style(9)**) but replaces the *naming* rules. OpenBSD names are lowercase with under-scores and carry no type information. This specification uses camelCase and Hungarian notation (see [1]).

Where this document is silent, follow **style(9)** and imitate the neighbouring code. This style is meant for your own projects. Patches submitted to OpenBSD itself must follow upstream **style(9)**, not this document.

2 Layout (inherited from KNF)

These rules are unchanged from **style(9)**.

- Indent with tabs, 8 columns wide. Continuation lines are indented by 4 spaces relative to the statement they continue.
- Maximum line length is 80 columns.
- The opening brace of a function is on its own line. All other opening braces are on the same line as the statement, and **else** stays on the line of the closing brace: **} else {**.
- In a function definition, the return type is on its own line, and the function name starts at column 0.
- Put a space after the keywords **if**, **for**, **while**, **switch** and **return**. Put no space between a function name and its opening parenthesis, and none after a cast.
- Binary operators have a space on both sides. Unary operators have none.
- **case** labels are aligned with their **switch**.
- Do not use braces around a single-statement body unless the statement spans more than one line.
- Parenthesize returned values: **return (x);**.
- Compare explicitly: **if (p == NULL)**, **if (*sz == '\0')**.
- Declarations come at the start of a block. Do not use **typedef** for structs.
- Comments use **/* ... */** only. Multi-line comments open with a lone **/***, use ***** on each following line and close with ***/**.

- Include order: `<sys/types.h>` first, then other `<sys/*.h>`, a blank line, `<net/*.h>`, a blank line, remaining system headers in alphabetical order, a blank line, then local "... headers.
- Header guards have the form `_NAME_H_`.
- Multi-statement macros are wrapped in `do { ... } while (0)` and all arguments are parenthesized.

3 Naming

3.1 General rules

Entity	Rule	Example
Local variable	Hungarian prefix, then Pascal-Case qualifier	<code>iRetryCount</code>
Function parameter	Same as local variable	<code>pszKey</code>
File-scope static	<code>s_</code> + variable name	<code>s_fVerbose</code>
Global variable	<code>g_</code> + variable name	<code>g_iRetryCount</code>
Struct member	struct prefix + <code>_</code> + variable name	<code>ce_iHits</code>
Function	camelCase, subsystem prefix first	<code>cacheLookup</code>
Static function	camelCase, subsystem prefix optional	<code>dumpEntry</code>
Struct / union / enum tag	camelCase, no typedef	<code>struct cacheEntry</code>
Macro, constant, enum value	UPPER_SNAKE with prefix	<code>CACHE_MAX_ENTRIES</code>

The underscore is permitted in camelCase names only to separate a scope prefix (`g_`, `s_`) or a struct-member prefix (`ce_`) from the rest of the name. Everywhere else it is forbidden, except in macros.

3.2 Hungarian prefix grammar

A variable name has the form

`[scope_] [pointer]* [type] Qualifier`

Everything before the qualifier is lowercase, and the qualifier starts with an uppercase letter. That capital letter is how a reader finds the boundary between the type prefix and the meaning. For example, `s_pszHostName` reads: file-static (`s_`), pointer (`p`), NUL-terminated string (`sz`), meaning "host name".

For a pointer to a struct, omit the type slot and name the struct in the qualifier: `pCache` is a `struct cache *`. A pointer to a pointer uses `pp`.

3.3 Type prefixes

Prefer *semantic* prefixes (what the value *means*: a count, a flag, a byte size) over purely mechanical ones (what the compiler calls it). This follows Simonyi's original intent and avoids the redundancy that gave Hungarian notation a bad name [1].

Prefix	Meaning	Example
c	single character	cDelim
i	int	iRetryCount
u	unsigned int	uFlags
l	long	lOffset
ul	unsigned long	ulMask
f	boolean / flag	fVerbose
n	count of items	nEntries
cb	count of bytes (<code>size_t</code>)	cbKeyLen
sz	NUL-terminated string	szHostName
a	array (combine with element type)	aiCounts
fd	file descriptor	fdSocket
t	time value	tExpire
e	enum value	eState
fn	function pointer	fnCallback
h	opaque handle	hSession
p	pointer (repeat per level)	pszKey, ppszArgv

Exemptions. These conventional names need no prefix: loop indices `i`, `j`, `k`; the getopt result `ch`; `argc` and `argv`; and any name required by an external API, `libc` or the kernel (`errno`, `environ`).

3.4 Functions

- Use camelCase in subsystem–verb–noun order: `cacheLookup`, `cacheFree`, `pfTestRule`.
- The subsystem prefix replaces Java’s package and class name. C has one flat global namespace, so every non-static function carries it.
- Functions carry no Hungarian prefix. The return type is visible on the line above the name.
- Driver entry points keep OpenBSD’s fixed vocabulary in camelCase: `fooMatch`, `fooAttach`, `fooDetach`, `fooIntr`, `fooIoctl`, `fooStart`, `fooInit`, `fooStop`.
- Userland programs still define `static void usage(void)`.

3.5 Things that stay unchanged

The following remain lowercase, as in OpenBSD, because they are visible outside the language: file names (`cache.c`), program names (`foo`, `food`, `fooctl`), man pages (`foo(8)`), configuration keys and sysctl names (`kern.hostname`). Macros and enum constants stay UPPER_SNAKE.

4 Translation table

OpenBSD	This specification	Note
last_seen	tLastSeen	time value
peer_addr	pPeerAddr	struct sockaddr *
vflag	fVerbose	option flag
nentries	nEntries	item count
buflen	cbBuf	bytes
sc_dev	sc_pDev	struct-member prefix kept
uvm_map_lookup_entry	uvmMapLookupEntry	function
pf_test_rule	pfTestRule	function
imsg_compose	imsgCompose	function
MAXPATHLEN	MAXPATHLEN	unchanged

5 Worked example

```
/*
 * cache.c - fixed-size lookup cache
 */

#include <sys/types.h>

#include <err.h>
#include <stdlib.h>
#include <string.h>

#include "cache.h"

#define CACHE_MAX_ENTRIES      64

struct cacheEntry {
    char          *ce_pszKey;
    size_t        ce_cbKeyLen;
    int           ce_iHits;
};

struct cache {
    struct cacheEntry *ca_pEntries;
    size_t            ca_nEntries;
    size_t            ca_nMaxEntries;
};

int                g_iRetryCount = 3;
static int         s_fVerbose;

struct cache *
cacheNew(size_t nMaxEntries)
{
    struct cache    *pCache;

    if ((pCache = calloc(1, sizeof(*pCache))) == NULL)
        return (NULL);
    pCache->ca_pEntries = calloc(nMaxEntries,
        sizeof(*pCache->ca_pEntries));
    if (pCache->ca_pEntries == NULL) {
        free(pCache);
        return (NULL);
    }
    pCache->ca_nMaxEntries = nMaxEntries;
}
```

```

        return (pCache);
    }

    struct cacheEntry *
    cacheLookup(struct cache *pCache, const char *pszKey)
    {
        size_t    i;

        for (i = 0; i < pCache->ca_nEntries; i++) {
            if (strcmp(pCache->ca_pEntries[i].ce_pszKey, pszKey) == 0) {
                pCache->ca_pEntries[i].ce_iHits++;
                return (&pCache->ca_pEntries[i]);
            }
        }
        return (NULL);
    }

    void
    cacheFree(struct cache *pCache)
    {
        size_t    i;

        if (pCache == NULL)
            return;
        for (i = 0; i < pCache->ca_nEntries; i++) {
            if (s_fVerbose)
                warnx("free %s", pCache->ca_pEntries[i].ce_pszKey);
            free(pCache->ca_pEntries[i].ce_pszKey);
        }
        free(pCache->ca_pEntries);
        free(pCache);
    }

```

The listing shows tabs as 8 spaces. Real source files use tab characters.

6 Tooling

A formatter handles layout, and a linter handles naming. The two configuration files in the appendices enforce most of this document. Neither tool can verify that a Hungarian *type* prefix is semantically correct, so that remains a review task.

A .clang-format

```

BasedOnStyle: LLVM
Language: Cpp
UseTab: Always
TabWidth: 8
IndentWidth: 8
ContinuationIndentWidth: 4
ColumnLimit: 80
BreakBeforeBraces: Custom
BraceWrapping:
  AfterFunction: true
  AfterControlStatement: Never
  AfterStruct: false
  AfterUnion: false
  AfterEnum: false
  BeforeElse: false
AlwaysBreakAfterReturnType: AllDefinitions
IndentCaseLabels: false

```

```
AllowShortIfStatementsOnASingleLine: Never
AllowShortLoopsOnASingleLine: false
AllowShortFunctionsOnASingleLine: None
PointerAlignment: Right
SpaceAfterCStyleCast: false
SpaceBeforeParens: ControlStatements
AlignAfterOpenBracket: DontAlign
SortIncludes: false
```

Recent clang-format releases rename `AlwaysBreakAfterReturnType` to `BreakAfterReturnType`. Use whichever your version accepts.

B .clang-tidy

```
Checks: '-*,readability-identifier-naming'
CheckOptions:
- key: readability-identifier-naming.FunctionCase
  value: camelBack
- key: readability-identifier-naming.GlobalFunctionCase
  value: camelBack
- key: readability-identifier-naming.VariableCase
  value: camelBack
- key: readability-identifier-naming.ParameterCase
  value: camelBack
- key: readability-identifier-naming.GlobalVariablePrefix
  value: g_
- key: readability-identifier-naming.StaticVariablePrefix
  value: s_
- key: readability-identifier-naming.MacroDefinitionCase
  value: UPPER_CASE
- key: readability-identifier-naming.EnumConstantCase
  value: UPPER_CASE
```

The clang-tidy `readability-identifier-naming` check can also enforce Hungarian prefixes through its `HungarianNotation.*` options. Check the documentation for your clang-tidy version before relying on them.

References

- [1] Hungarian notation, Wikipedia. https://en.wikipedia.org/wiki/Hungarian_notation
- [2] OpenBSD `style(9)`, kernel source file style guide. `man 9 style`